Extending HTTP Authentication

Joe Gregorio,
President, BitWorking, Inc.

Background

In the session just before this one you saw a review of the current state of weblogging APIs. All of these APIs have different authentication schemes and most send passwords in plain text. The AtomAPI aims to be a unified blogging API and as such needs to have an authentication mechanism that can be used across the board, from large dedicated blogging hosts such as TypePad, LiveJournal, and Blogger down to the personal installs of Movable Type and Blosxom. Now given the goal of the AtomAPI to make better use of the HTTP protocol it seems natural to try to use the native authentication mechanisms of HTTP.

"Just" use Digest

> Sign in bike shop: "Could You Just... costs 40 Kroner"[1]

HTML comes with two authentication mechanisms, Basic and Digest, both defined in RFC 2617[2]. Basic authentication passes everything in plain text and is clearly not secure so we will focus on using Digest authentication. Now the high end dedicated blog hosting providers are pretty easy to satisfy as they have total control over their machines. Editing `httpd.conf` and enabling modules are available to them so just using Digest authentication or even Basic authentication over SSL are viable options for them. The situation gets trickier as you move down to the average hosted user running a blog. With a typical low cost hosting account you:

1. Can't make use of `.htaccess`
2. Can't make use of `httpd.conf`
3. Digest may not even be turned on for the account.

Without access to `.htaccess` files these people are effectively locked out of using Basic and Digest Authentication. While this list does appear restrictive, it is the reality faced by many bloggers today.

Now at first blush this might not seem too much of a problem. Just handle the authentication from the CGI, right? Let's look at what happens when you try that. Without going into a lot of detail right now let's look at a wire trace when a simple CGI script tries to handle Digest authentication on it's own under Apache 2.0.

Here is a simple CGI script written in Python that generates a 401 Unauthorized header when it gets a request, this should trigger the browser to request authentication credentials, which in turn should be returned to the CGI script.

```
import os
print "Status: 401 Unauthorized"
print 'WWW-Authenticate: Digest realm="testrealm@host.com"\
    , qop="auth,auth-int", nonce="dcd9...c093"\
    , opaque="5ccc...f40e41" '
print "Content-type: text/plain\n\n"

print "\n".join(["%s=%s" % (key, value) for key,value in os.environ.iteritems()])
```

Let's look at a wire-trace to see what really happens. Given this request:

```
GET /cgi-bin/test.py HTTP/1.1
Host: 127.0.0.1:8085
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
    Gecko/20030516 Mozilla Firebird/0.6
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,
   text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate,compress;q=0.9
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

This is the response:

```
HTTP/1.1 401 Unauthorized
Date: Mon, 13 Oct 2003 03:44:46 GMT
Server: Apache/2.0.45 (Win32)
WWW-Authenticate: Digest realm="testrealm@host.com", qop="auth,auth-int",
    nonce="dcd98b...b0c093", opaque="5ccc...40e41"
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/plain; charset=ISO-8859-1

HTTP_ACCEPT_CHARSET=ISO-8859-1,utf-8;q=0.7,*;q=0.7
.
.
.
DOCUMENT_ROOT=C:/Apache/Apache2/htdocs
```

So my user-agent pops up and asks for my name and password and then sends this:

```
GET /cgi-bin/test.py HTTP/1.1
Host: 127.0.0.1:8085
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
   Gecko/20030516 Mozilla Firebird/0.6
Accept: text/xml,application/xml,application/xhtml+xml,
  text/html;q=0.9,text/plain;q=0.8,video/x-mng,image/png,
  image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate,compress;q=0.9
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Authorization: Digest username="joe", realm="testrealm@host.com",
   nonce="dcd98...b0c093", uri="/cgi-bin/test.py", response="21f...e828",
   opaque="5cc...e41", qop=auth, nc=00000001, cnonce="48...403"
```

And the server responds with:

```
HTTP/1.1 401 Unauthorized
Date: Mon, 13 Oct 2003 03:44:52 GMT
Server: Apache/2.0.45 (Win32)
WWW-Authenticate: Digest realm="testrealm@host.com", qop="auth,auth-int",
    nonce="dcd...093", opaque="5cc...e41"
Keep-Alive: timeout=15, max=99
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/plain; charset=ISO-8859-1

HTTP_ACCEPT_CHARSET=ISO-8859-1,utf-8;q=0.7,*;q=0.7
SERVER_SOFTWARE=Apache/2.0.45 (Win32)
SERVER_PROTOCOL=HTTP/1.1
SCRIPT_FILENAME=C:/Apache/Apache2/cgi-bin/test.py
SERVER_ADMIN=admin@this.com
SCRIPT_NAME=/cgi-bin/test.py
SERVER_SIGNATURE=<address>Apache/2.0.45 (Win32) Server at 127.0.0.1 Port 8085</address>
REQUEST_METHOD=GET
HTTP_HOST=127.0.0.1:8085
HTTP_KEEP_ALIVE=300
QUERY_STRING=
REQUEST_URI=/cgi-bin/test.py
HTTP_ACCEPT=text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,
    text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
HTTP_USER_AGENT=Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
    Gecko/20030516 Mozilla Firebird/0.6
HTTP_CONNECTION=keep-alive
SERVER_NAME=127.0.0.1
REMOTE_ADDR=127.0.0.1
REMOTE_PORT=1215
HTTP_ACCEPT_LANGUAGE=en-us,en;q=0.5
SERVER_PORT=8085
GATEWAY_INTERFACE=CGI/1.1
HTTP_ACCEPT_ENCODING=gzip,deflate,compress;q=0.9
SERVER_ADDR=127.0.0.1
DOCUMENT_ROOT=C:/Apache/Apache2/htdocs
```

Note that **none** of the authentication information in the `Authorization:` header made it through. The problem we run into is that Apache never passes the information in the `Authorization:` header into the CGI program. If you don't get the information in that header then there's no hope of doing Digest authentication on your own.

Now this is in itself not an insurmountable problem, there are in fact several ways to fix this.

1. Use Basic or Digest the way Apache wants you to, which unfortunately requires the module be turned on, and also requires .htaccess or httpd.conf access.

2. Run your CGI program as an Apache module. Apache modules get all the headers, including the `Authorization:` header.

3. Recompile Apache using a flag that tells it to pass along all authentication information to the CGI program.

Obviously these aren't available to the average blogger.

Now from the opposite side of the fence are the blog tool authors. Systems like MT don't defer to an outside source for their authentication, they want to do all the authentication themselves. As we just saw Apache tries to do the Authentication itself, and if it can't, it fails to pass the authentication information down to the CGI program.

The Requirements

So reviewing the constraints of both the clients and servers, we get the following list of requirements for any acceptable authentication scheme.

1. Is a challenge-response digest authentication scheme.
2. The authentication can be handled by a CGI program with no `.htaccess`, `httpd.conf` or requirements to be running as an Apache module, ala mod_perl or mod_python.
3. Gracefully extends current authentication schemes.
4. The simplest thing that could possibly work[3].
5. Works with the current systems in the field, that is, it offers an easy migration path.
6. Doesn't require the system, client or server, to store plain text passwords.

Note that what is presented here is a technical solution to this problem. Note that other solutions from other domains may bubble up over time. On the sociological front, for example, a series of high profile blogs getting hacked may raise the security issues with current blogging tools to a higher level and the customers may force a solution. In the realm of market forces, hosting providers, in a bid to win more customers may decide to compete on security and thus start offering sites with preconfigured Digest authentication turned on. Those are interesting possibilities, but the rest of this session will focus on the technical solution.

A Review of Digest Authentication

First, let's do a quick review of Digest authentication. Here is a wire-trace of a single transaction. First we setup a `.htaccess` file to turn on digest for a single file.

```
<Files test_digest.html>
    AuthType Digest
    AuthName test
    AuthDigestFile "c:\apache\apache2\digest_passwords"
    Require valid-user
</Files>
```

Then we create the password file referenced in the above `.htaccess` file.

```
C:\Apache\Apache2>bin\htdigest.exe -c digest_passwords test joe
Adding password for joe in realm test.
New password: ****
Re-type new password: ****
```

First comes the initial Request

```
GET /test_digest.html HTTP/1.1
Host: localhost:8085
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
    Gecko/20030516 Mozilla Firebird/0.6
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,
   text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate,compress;q=0.9
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
```

The initial Response is a 401 Unauthorized with a challenge in the form of the `WWW-Authenticate:` header.

```
HTTP/1.1 401 Authorization Required
Date: Tue, 14 Oct 2003 01:12:13 GMT
Server: Apache/2.0.45 (Win32)
WWW-Authenticate: Digest realm="test", nonce="MJc5V...ba36", algorithm=MD5,
   qop="auth"
Content-Length: 479
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1
```

The client then prompts for credentials and re-submits the request with an Authorization header:

```
GET /test_digest.html HTTP/1.1
Host: localhost:8085
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
   Gecko/20030516 Mozilla Firebird/0.6
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,
    text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate,compress;q=0.9
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Authorization: Digest username="joe", realm="test", nonce="MJc5V...5ba36",
     uri="/test_digest.html", algorithm=MD5, response="2772...3fbd",
     qop=auth, nc=00000001, cnonce="082...740"
```

The server authenticates against the information and now we get our response.

```
HTTP/1.1 200 OK
Date: Tue, 14 Oct 2003 01:12:18 GMT
Server: Apache/2.0.45 (Win32)
Authentication-Info: rspauth="9a9...c0d", cnonce="082...740", nc=00000001, qop=auth
Last-Modified: Tue, 14 Oct 2003 01:11:09 GMT
ETag: "29efe-39-5261f7cb"
Accept-Ranges: bytes
Content-Length: 57
Keep-Alive: timeout=15, max=99
Connection: Keep-Alive
Content-Type: text/html; charset=ISO-8859-1

<html>
    <body>
        <h1>Success!</h1>
    </body>
</html>
```

Here the details of how the Digest authentication scheme works:

1. The client tries to do something that requires authentication, for instance doing a GET on `http://localhost:8085/test_digest.html`. The server sends back an HTTP error code of "401 Unauthorized", and a `WWW-Authenticate` header like this:

   ```
   WWW-Authenticate: Digest realm="test", nonce="MJc...a36",   algorithm=MD5, qop="auth"
   ```

   So what are all those fields and what to do they mean?

   realm
   : A string to be displayed to users so they know which user name and password to use. It is a server assigned name that represents a protection space. Note that a server may send multiple challenges with the same authentication scheme and different realms.

   nonce
   : A unique server generated string, preferably base64 or hexadecimal data. Nonces in general a

quantities that vary over time and are used to prevent replay attacks.

algorithm
: What algorithm to use when constructing the digest.

qop
: The quality of protection value. Is set to "auth" to indicate authentication, or "auth-int" for authentication with integrity protection. If `qop` is set to "auth-int" then the body is used in calculating the digest.

These are the values we are covering for now, there are other optional values that can be included, but were left out for now to simplify the discussion. The values not discussed are:

domain
: A list or URIs that define the protection space. If these aren't supplied then the client may assume that the authentication is good for the entire server. Note that the list returned may include absolute URIs, even URIs that are under different servers.

opaque
: An opaque string that the client should send back unchanged. This is a handy little bucket for storing any state information you want to use.

stale
: Can have the value of "true" or "false" and indicates if the last authentication request the client sent used a stale `nonce`.

2. The client takes the username, the realm given by the server, and the password, and concatenates them to create an intermediate value which we will call A1:

```
A1 = username + ":" + realm + ":" + password
```

3. The client takes the HTTP verb it wants to use (in this case "GET") and the path part of the URL it wants to post to (in this case "/test_digest.html"), and concatenates them into an intermediate value which we will call A2:

```
A2 = verb + ":" + uri
```

4. The client creates a unique client-specific value, which we will call "cnonce". How this happens is completely client-specific, but it should change on every request, and future values should not be guessable.

5. The client takes A1, A2, chooses a qop given by the server, the nonce given by the server, and the cnonce created by the client, and creates a digest, which we will call "response":

```
response = md5(md5(A1) + ":" + nonce + ":" + "00000001" + ":" +
    cnonce + ":" + qop + ":" + md5(A2))
```

Note that we are using the md5 hash of A1, and that this is the only item that relies upon the password. What this means is that md5(A1) is all you need to store, you do not need to store plain text passwords to use Digest. This is also true for all the following variations on Digest in the rest of this paper, that plain text passwords need not be stored on the client or the server.

6. The client resends its original request, with the addition of the `Authentication` header with all of the following values filled in:

```
Authorization: Digest username="joe", realm="test", nonce="MJc5V...ba36",
   uri="/test_digest.html", algorithm=MD5, response="2772...3fbd", qop=auth,
   nc=00000001, cnonce="082c..740"
```

The only value we haven't already talked about is `nc`. This is a count of the number of times we've

used the challenge nonce. This is used to prevent replay attacks.

Note that the client sends back to the server the 'response' value and everything it needs to calculate 'response' except for the value of A1, which the server should know.

Also note the things that this authentication is holding invariant, i.e. things that can't be changed or altered by a malicious intermediary. The verb, path, nonce, nonce count, quality of protection, name, realm and password are all wrapped up in the calculation of the 'response'. If a malicious intermediary along the way changes any one of those values then the authentication will fail. Also note that if the **qop** is chosen to be “auth-int” then the calculation of the response also includes the message body.

7. The server checks the validity of the request by recalculating the 'response' value and if the two match then the request is considered valid. If everything checks out OK then the server returns the Response and may include the `Authentication-Info:` header in the response. Here are two example Authentication-Info: headers:

```
Authentication-Info: nextnonce="082...c0"

Authentication-Info: rspauth="9a9...10d", cnonce="082c...a740", nc=00000001, qop=auth
```

   This first response can contain the next nonce value to use for the next request, thus obviating multiple round trips for authentication challenges. If it doesn't include `nextnonce` then the next request should just reuse the old nonce and increment the value of `nc`.

   The second response contains some familiar parameters, `cnonce`, `nc` and `qop`. What is happening here is that the server is authenticating itself to the client, proving that it too knows the shared secret. The calculation of `rspauth` is the same as for `response` except that the value of A2 drops the inclusion of the verb and is calculated as:

```
A2 = ":" + uri
```

   If this change wasn't made then you would have `response=rspauth` and that wouldn't be very helpful in trying to authenticate the server.

8. If the username/password is not valid, the server will respond with an HTTP error code 403 and a new `WWW-Authenticate` header starting the process all over again. If the client screwed something up (forgot a value, sent a malformed authentication request), the server will respond with an HTTP error code 400 and a new `WWW-Authenticate` header.

   If the client successfully authenticated, the server will do what the client asked (in this case GET a file). Every subsequent response from the server may contain an `Authentication-Info` header that includes a "nextnonce" value. If present, the client should discard the previous nonce value and use the new nonce value to recalculate the digest response on the next request. (This protects against replay attacks.) The client should cache and reuse the other values given by the server: `realm`, `qop`, `algorithm`. **Either way, only one extra round trip is required per session before the first action, to get the initial authentication challenge. The client does not need to do additional round trips once they have successfully authenticated, as long as they stay current with their nonce values.**

   If the server does not return a new nonce value, the client should continue using the old one and increment the value of `nc` as a hexadecimal number and recalculate the digest response. So on the second request, the client would recalculate the response like this:

```
response = md5(md5(A1) + ":" + nonce + ":" + "00000002" + ":" + cnonce + ":" + qop +
":" + md5(A2))
```

First Steps Toward a Solution

A reminder of how we got here: in most cases a CGI hosted user with Digest not turned on will not get the Authorization: header. So we had to find a way to do roughly the same thing as Digest, but to change just enough that the headers got through to the CGI program. So Mark Pilgrim and I made the following changes to Digest:

1. Used SHA1 instead of MD5.
2. Triggered authentication by rejecting a request with an HTTP status code of 447 instead of 401.
3. The server response when rejecting a request with a 447 also included an Atom-Authenticate: header that mimics the `WWW-Authenticate:` header in HTTP Digest.
4. The client sends an `Atom-Authorization:` header with all the Digest authentication information in it, mimicking the `Authorization:` header in HTTP Digest.
5. The server sends back an `Atom-Authentication-Info:` header with the 'nextnonce', taking the place of the `Authentication-Info:` header of HTTP Digest.

Here is an example of this authentication in action:

```
HTTP/1.1 447 Authorization Required
Date: Tue, 14 Oct 2003 01:12:13 GMT
Server: Apache/2.0.45 (Win32)
Atom-Authenticate: Digest realm="test", nonce="Jc5V...5ba36", algorithm=SHA, qop="auth"
Content-Length: 479
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1
```

The client then prompts for credentials and re-submits the request with an Authorization header:

```
GET /test_digest.html HTTP/1.1
Host: localhost:8085
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
    Gecko/20030516 Mozilla Firebird/0.6
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,
    text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate,compress;q=0.9
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Atom-Authorization: Digest username="joe", realm="test", nonce="Jc5...a36",
    uri="/test_digest.html", algorithm=SHA, response="277...fbd",
    qop=auth, nc=00000001, cnonce="082...740"
```

The server authenticates against the information and now we get our response.

```
HTTP/1.1 200 OK
Date: Tue, 14 Oct 2003 01:12:18 GMT
Server: Apache/2.0.45 (Win32)
Atom-Authentication-Info: nextnonce="0a9...8c0"
Last-Modified: Tue, 14 Oct 2003 01:11:09 GMT
ETag: "29efe-39-5261f7cb"
Accept-Ranges: bytes
Content-Length: 57
Keep-Alive: timeout=15, max=99
Connection: Keep-Alive
Content-Type: text/html; charset=ISO-8859-1

<html>
    <body>
        <h1>Success!</h1>
    </body>
</html>
```

Several things to note:

1. This works. This scheme was tested and indeed the client and server can authenticate using a CGI program.

2. **This is wrong.** Don't do this.

The Corrected Solution

Now this was a good first pass but we did several things wrong as Ken Coar pointed out on his site[4] and on the atom-syntax mailing list[5]. Basically it boils down to the fact that we ignored the built-in extensibility of the HTTP authentication mechanism and went around it starting with returning an HTTP status code of 447.

What we should have done:

1. Reject the request with an HTTP status code of 401.
2. The server response includes an `Authenticate:` header that includes Atom as an authentication scheme.
3. The client then sends an `Authorization:` header with the scheme of Atom with all the authentication information going into `X-Atom-Authorization:` header.
4. With every request the server sends back an `X-Atom-Authentication-Info:` header with the 'nextnonce'.

Note that this now uses the extensibility of the HTTP authentication scheme. When the server responds with a `WWW-Authenticate:` header, that header lists, in order of preference, the authentication schemes that the server supports, or alternately the response may have multiple WWW-Authenticate: headers each with a different challenge. The client then selects the strongest scheme it knows. Note that the whole authentication mechanism is designed to be robust to the addition of new/unknown schemes. We are leveraging this extensibility by passing in an authentication scheme to Apache that it doesn't know about. The Apache server reacts appropriately by passing the request onto the CGI program to give it a chance to handle the authentication.

Here the details of how the Atom authentication scheme works:

1. The client tries to do something that requires authentication, for instance, POSTing a new entry to `http://diveintomark.org/cgi-bin/atom.cgi/blog_id=14`. The server sends back and HTTP error code of "401 Unauthorized", and a `WWW-Authenticate` header like this:

   ```
   WWW-Authenticate: Atom realm="some server realm name", qop="atom-auth",
      algorithm="SHA", nonce="some unique server-specific value"
   ```

2. The client takes the username, the realm given by the server, and the password, and concatenates them to create an intermediate value which we will call A1:

   ```
   A1 = username + ":" + realm + ":" + password
   ```

3. The client takes the HTTP verb it wants to use (in this case "POST") and the path part of the URL it wants to post to (in this case "/cgi-bin/atom.cgi/blog_id=14"), and concatenates them into an intermediate value which we will call A2:

   ```
   A2 = verb + ":" + uri
   ```

4. The client creates a unique client-specific value, which we will call "cnonce". How this happens is completely client-specific, but it should change on every request, and future values should not be guessable.

5. The client takes A1, A2, the qop given by the server, the nonce given by the server, and the cnonce created by the client, and creates a digest, which we will call "response":

```
response = sha(sha(A1) + ":" + nonce + ":" + "00000001" +
   ":" + cnonce + ":" + qop + ":" + sha(A2))
```

6. The client resends its original request, with the addition of two headers. First the `X-Atom-Authentication:` header with all of the following values filled in:

```
X-Atom-Authentication: Atom username="...", realm="...",
   nonce="...", uri="...", qop="atom-auth", nc="00000001",
   cnonce="...", response="..."
```

   And then the Authentication header signaling that we are using the Atom authentication scheme:

```
Authorization: Atom key="value"
```

   Note that the key-value pair after the 'Atom' has no significance for us as Apache will not pass down the `Authentication:` header to the CGI program even when it is an authentication scheme it doesn't know, which is why all the real useful information was moved into `X-Atom-Authentication:` which *will* get passed to a CGI program.

7. The server authenticates the information just like it did in the Digest scheme, by recalculating 'response' and making sure it matches.

Here is an example of just such a transaction:

```
HTTP/1.1 401 Authorization Required
Date: Tue, 14 Oct 2003 01:12:13 GMT
Server: Apache/2.0.45 (Win32)
WWW-Authentice: Atom realm="test", nonce="Jc5...5ba36", algorithm=SHA, qop="auth"
Content-Length: 479
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1
```

The client then prompts for credentials and re-submits the request with an Authorization header:

```
GET /test_digest.html HTTP/1.1
Host: localhost:8085
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4b)
    Gecko/20030516 Mozilla Firebird/0.6
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,
    text/plain;q=0.8,video/x-mng,image/png,image/jpeg,image/gif;q=0.2,*/*;q=0.1
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate,compress;q=0.9
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Authorization: Atom key="value"
X-Atom-Authorization: Atom username="joe", realm="test", nonce="MJc5...5ba36",
    uri="/test_digest.html", algorithm=SHA, response="277...20bd", qop=auth,
    nc=00000001, cnonce="082c...ca740"
```

The server authenticates against the information and now we get our response.

```
HTTP/1.1 200 OK
Date: Tue, 14 Oct 2003 01:12:18 GMT
Server: Apache/2.0.45 (Win32)
X-Atom-Authentication-Info: nextnonce="23490...a102e"
Last-Modified: Tue, 14 Oct 2003 01:11:09 GMT
ETag: "29efe-39-5261f7cb"
Accept-Ranges: bytes
Content-Length: 57
Keep-Alive: timeout=15, max=99
Connection: Keep-Alive
Content-Type: text/html; charset=ISO-8859-1

<html>
    <body>
        <h1>Success!</h1>
    </body>
</html>
```

Please note that:

1. This works. A GUI Python client and a Python based server CGI script were written to demonstrate this scheme.

2. **Do this**. As opposed to our first attempt, this scheme correctly extends HTTP authentication because it:

    1. Uses the correct status code (401).

    2. Signals a new authentication scheme by changing the 'auth-scheme' token, like we did in this case, from “Digest” to “Atom”.

    3. Uses custom headers, but only the bare-minimum required to get the job done.

Multiple Challenges

It is possible for the server to challenge with multiple authentication schemes. One way to do this is include multiple WWW-Authenticate: headers in the challenge response:

```
HTTP/1.1 401 Authorization Required
Date: Tue, 14 Oct 2003 01:12:13 GMT
Server: Apache/2.0.45 (Win32)
WWW-Authenticate: Digest realm="test", nonce="MJc...b36", algorithm=MD5, qop="auth"
WWW-Authenticate: Digest realm="other_realm", nonce="MJc...a36", algorithm=MD5, qop="auth"
WWW-Authenticate: Atom realm="test", nonce="V3ed...9901", algorithm=SHA, qop="auth"
WWW-Authenticate: Atom realm="other_realm", nonce="V3e...901", algorithm=SHA, qop="auth"
Content-Length: 479
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=iso-8859-1
```

Migration Issues

Here is the list of requirements again.

1. Is a challenge-response digest authentication scheme.
2. Is able to be handled by a CGI program with no `.htaccess`, `httpd.conf` or requirements to be running as an Apache module, ala mod_perl or mod_python.
3. Gracefully extends current authentication schemes.
4. Is the simplest thing that could possibly work.
5. Current working systems can migrate to the new system.
6. Doesn't require the system, client or server, to store plain text passwords.

What I would like to focus on are items 5 and 6. These seem like perfectly reasonable requests. Unfortunately they aren't really compatible.

Current systems store either plain text passwords or they don't. A rather simple minded classification, but it's the truth, either they stored the passwords in plain text, which is not secure, or they store the password in a hashed format. For example, Movable Type stores passwords using the crypt() function[6]. Also remember that Digest and Atom authentication do not require you to store plain text passwords. Digest scheme allows you to just store `md5(A1)` and Atom authentication allows you to store `sha1(A1)`.

Any scheme that doesn't require you to store plain text passwords will require the password to be stored in some sort of hash, either by itself or hashed against other values. The hashes we are talking about here are one-way hashes. That's important. One-way. They can't be reversed to get back the original password.

If you were going to migrate to using the Atom authentication scheme and didn't store you passwords as a SHA1 of `name:realm:password` then you have a problem. Similarly if you were to begin using Digest authentication and didn't store your passwords as an MD5 of `name:realm:password` then you again have a problem. Because unless you stored them in this manner, or happened to store them as plain text then you've got a migration problem. That is, your users are going to have to re-enter their passwords when you enable the new authentication mechanism.

Note that this is not just a problem in the HTTP side. You will also run into this is a problem if you want to connect to any other authentication system. For example, if you are going to use LDAP as your system to manage user authentication, then you would again have a problem unless you stored the SHA1 of your user passwords. Now image you want to use Digest authentication over HTTP but check that authentication against an LDAP server.

Even if everyone agreed on **one** way of storing their passwords locally it still wouldn't solve the problem. What was once secure yesterday may be considered completely insecure tomorrow. If that occurs to your **one true way** to store passwords, then you end up with the migration issues outlined above, which is that every time a new authentication mechanism is added the user will have to re-enter their password. The solution is to not have **a** way to store passwords in your system, instead you should be able to store multiple passwords for each user, one hashed password for each possible authentication scheme you will be using as either a client or a server, and forget **never** having to re-enter passwords as it's a completely unreasonable design goal.

Choosing SHA1 over MD5

One of the things we changed from Digest to Atom that we really didn't have to change was the algorithm to use. Part of the reason why we moved to SHA1 over MD5 was the availability of SHA1 for all the languages we checked. In addition we had heard that MD5 was aging and wasn't really considered as secure as it was when RFC 2617 was published. In the paper "On the security of dedicated hash functions" by Rompay, Preneel and Vandewalle[7] the authors consider MD5:

> "In 1996 the attack on MD4 was extended to MD5. It finds collisions for a random initial chaining variable and two message blocks differing on only one word; the are called *collisions for the compression function*. The attack can be seperated into three main parts: finding a collision for rounds ½, finding a collision for rounds ¾, and connecting these two. It takes a few hours on a PC. To obtain a real collision for the hash function the value of the initial chaining variable would have to be directed to the prescribed value. This would require hard additional cryptanalytic work, but because of the flexibility of the evaluation technique it is anticipated that this can be done. So it seems unwise to use MD5 as a collision resistant hash function."

Credits

Many many thanks to Ken Coar for working patiently with Mark and I on fixing our initial version into the better scheme described here in. Also special thanks to Mark Pilgrim for working out this system with me over the course of two midnight hacking sessions wherein we wrote both the client and the server and the corresponding blog articles. Any errors and omissions lie solely with myself.

[1] Could You Just, http://c2.com/cgi/wiki?CouldYouJust
[2] HTTP Authentication: Basic and Digest Access Authentication, http://ftp.ics.uci.edu/pub/ietf/http/rfc2617.txt
[3] Do The Simplest Thing That Could Possibly Work, http://xp.c2.com/DoTheSimplestThingThatCouldPossiblyWork.html
[4] Raining on the atomic parade, http://ken.coar.org/burrow/index?entry=568
[5] http://www.imc.org/atom-syntax/mail-archive/msg00334.html
[6] http://www.rayners.org/2003/09/11/updating_mt_to_play_well_with_atom.php
[7] http://citeseer.ist.psu.edu/cache/papers/cs/2504/http:zSzzSzwww.esat.kuleuven.ac.bezSz~vrompayzSzpaper_wic98.pdf/on-the-security-of.pdf/

This presenation has ended.
Click anywhere to return.

_

